

Chương 1 đã trình bày bối cảnh, động lực và mục tiêu của đề tài: xây dựng một lớp điều phối phi tập trung nhằm hỗ trợ vận hành Messaging Layer Security (MLS) trên môi trường mạng ngang hàng. Để làm rõ cơ sở khoa học cho hướng tiếp cận này, Chương 2 tập trung trình bày các nền tảng lý thuyết cốt lõi của đề tài, bao gồm bài toán nhất quán trạng thái mật mã trong hệ phân tán, các hướng nghiên cứu liên quan, cơ chế hoạt động của MLS và TreeKEM, cùng các đặc tính của mạng ngang hàng phi tập trung.

Nội dung chương được tổ chức thành bốn phần chính. Mục 2.1 phân tích ngữ cảnh bài toán và chỉ ra vì sao việc đưa MLS lên mạng ngang hàng không chỉ là một bài toán truyền thông, mà là một bài toán nhất quán trạng thái mật mã. Mục 2.2 tổng hợp các kết quả nghiên cứu và giải pháp liên quan, từ các mô hình Sender Keys, Megolm đến MLS tập trung, DMLS và các hướng dựa trên đồng thuận phân tán. Mục 2.3 trình bày nền tảng MLS và TreeKEM, đặc biệt là khái niệm Epoch, Proposal, Commit, Delivery Service và bài toán tuần tự hóa thay đổi trạng thái. Mục 2.4 trình bày nền tảng mạng ngang hàng, go-libp2p, GossipSub và vai trò của đồng hồ logic trong việc sắp xếp thông điệp ứng dụng.

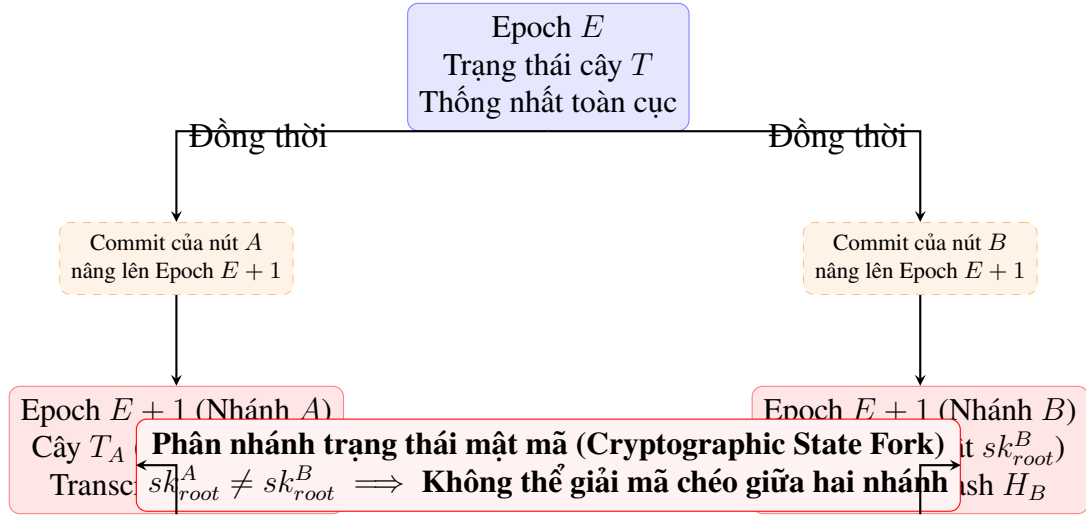
0.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán

Trong các hệ thống phân tán thông thường, nhất quán trạng thái là bài toán đảm bảo nhiều nút mạng cùng quan sát và cập nhật dữ liệu theo một quy tắc thống nhất. Đối với hệ thống nhắn tin mã hóa đầu-cuối nhóm, bài toán này trở nên phức tạp hơn vì trạng thái cần nhất quán không chỉ là dữ liệu ứng dụng, mà còn là trạng thái mật mã dùng để sinh khóa mã hóa, xác thực thành viên và bảo vệ lịch sử hội thoại.

MLS biểu diễn một nhóm bằng một trạng thái mật mã tiến hóa qua các kỷ nguyên, gọi là Epoch. Mỗi Epoch tương ứng với một tập thành viên hợp lệ và một tập bí mật nhóm được chia sẻ giữa các thành viên đó. Khi nhóm thay đổi thành viên hoặc khi thành viên cập nhật khóa, trạng thái nhóm phải chuyển từ Epoch hiện tại sang Epoch mới. Nếu tất cả thành viên cùng áp dụng một thay đổi theo cùng một thứ tự, nhóm tiếp tục duy trì khả năng giao tiếp. Ngược lại, nếu các thành viên áp dụng các thay đổi khác nhau hoặc áp dụng cùng các thay đổi theo thứ tự không tương thích, nhóm có thể rơi vào trạng thái phân nhánh mật mã.

Hiện tượng này được gọi là rẽ nhánh trạng thái mật mã (Cryptographic State Fork). Đây là tình huống trong đó hai tập thành viên cùng tin rằng mình đang ở Epoch hợp lệ tiếp theo của cùng một nhóm, nhưng thực tế lại nắm giữ các bí mật (secrets), hàm băm cây (tree hash) và lịch sử giao tiếp (transcript hash) khác nhau. Khi đó, thông điệp được mã hóa trên một nhánh sẽ không thể được xử lý hợp lệ trên nhánh còn lại. Khác với xung đột dữ liệu thông thường trong hệ phân tán, xung

đột trạng thái mật mã không thể được giải quyết đơn giản bằng cách “merge” hai phiên bản dữ liệu, vì mỗi nhánh đã phát sinh một chuỗi khóa và xác thực riêng. Hiện tượng rẽ nhánh trạng thái mật mã này được minh họa chi tiết trong Hình 0.1.



Hình 0.1: Minh họa hiện tượng rẽ nhánh trạng thái mật mã (Cryptographic State Fork)

0.1.1 Từ mô hình tập trung sang mô hình ngang hàng

Trong các hệ thống nhắn tin hiện đại, kể cả khi nội dung tin nhắn được mã hóa đầu-cuối, máy chủ trung tâm vẫn thường giữ vai trò quan trọng trong việc định tuyến, lưu trữ khóa công khai ban đầu, phân phối thông điệp và hỗ trợ thiết bị ngoại tuyến. Với MLS, vai trò này thường được mô hình hóa thông qua Delivery Service (DS) **rfc9750**. Trong các triển khai thông thường, DS là nơi nhận thông điệp từ client, phân phối thông điệp tới các thành viên còn lại và có thể tham gia áp đặt thứ tự đối với các thông điệp thay đổi trạng thái nhóm.

Mô hình tập trung giúp đơn giản hóa bài toán tuần tự hóa. Khi nhiều thành viên cùng gửi Commit tại cùng một Epoch, DS có thể đóng vai trò như một bộ sắp xếp thứ tự, quyết định Commit nào được chấp nhận trước và Commit nào trở thành dự thừa. Chính điểm này khiến MLS vận hành thuận lợi trong mô hình Client-Server: hệ thống có một vị trí tự nhiên để áp đặt thứ tự tuyến tính cho các thay đổi trạng thái.

Khi chuyển sang mạng ngang hàng, điểm tựa này biến mất. Các nút mạng có thể gửi thông điệp trực tiếp hoặc gián tiếp qua nhiều đường truyền khác nhau. Độ trễ truyền tin không đồng đều, thông điệp có thể đến khác thứ tự, nút mạng có thể offline đột ngột và mạng có thể bị chia cắt thành nhiều phân vùng. Trong môi trường như vậy, không còn một máy chủ trung tâm hiển nhiên để quyết định đâu là Commit “đến trước”. Đây là điểm gây kiến trúc khi đưa MLS từ mô hình triển khai tập trung thông thường sang mô hình P2P phi tập trung.

0.1.2 Mô hình hệ thống và giả định nghiên cứu

Trong phạm vi đề án, hệ thống được mô hình hóa như một mạng gồm nhiều nút ngang hàng, trong đó mỗi nút tương ứng với một thiết bị của người dùng. Các nút này có thể tham gia vào một hoặc nhiều nhóm liên lạc khác nhau. Mỗi nhóm được xác định bởi một định danh riêng, một tập thành viên hợp lệ và một trạng thái MLS được lưu cục bộ trên từng thiết bị. Thay vì phụ thuộc vào một máy chủ trung tâm luôn sẵn sàng để lưu trữ và phân phối toàn bộ thông điệp, các thiết bị trao đổi dữ liệu trực tiếp hoặc gián tiếp với nhau thông qua mạng ngang hàng.

Đặc điểm quan trọng của môi trường này là các nút không luôn quan sát được cùng một trạng thái tại cùng một thời điểm. Một thông điệp có thể đến trễ, đến sai thứ tự, bị lặp lại hoặc tạm thời không đến được do điều kiện mạng. Một thiết bị cũng có thể rời mạng, quay trở lại sau một khoảng thời gian ngoại tuyến, hoặc chỉ nhìn thấy một phần các thành viên đang hoạt động. Những hiện tượng này là bình thường trong mạng P2P, nhưng lại trở thành vấn đề nghiêm trọng khi áp dụng cho MLS, vì mỗi thay đổi trạng thái nhóm đều gắn với một epoch và một trạng thái mật mã cụ thể.

Do đó, đề án tập trung vào bài toán điều phối trạng thái MLS trong môi trường P2P không ổn định. Các tình huống chính được xem xét bao gồm độ trễ mạng không đồng đều, thiết bị ngoại tuyến, peer churn và network partition. Mục tiêu của mô hình là phản ánh các điều kiện vận hành thực tế mà một ứng dụng liên lạc ngang hàng có thể gặp phải, từ đó làm cơ sở cho việc thiết kế các cơ chế như Single-Writer, Epoch Checks và Fork Healing ở các chương sau.

Một giả định xuyên suốt của đề án là MLS vẫn giữ vai trò lõi mật mã chuẩn của hệ thống. Đề án không thay đổi thuật toán mã hóa, xác thực hay key schedule của MLS. Thay vào đó, phần đóng góp được đặt ở lớp điều phối phía trên MLS: kiểm soát thời điểm tạo Commit, kiểm tra epoch của thông điệp nhận được, phát hiện khi trạng thái nhóm bị phân nhánh và đưa các nút hội tụ trở lại sau khi mạng được kết nối. Cách tiếp cận này giúp hệ thống tận dụng được các bảo đảm mật mã của MLS/OpenMLS, đồng thời bổ sung những cơ chế cần thiết để vận hành trong môi trường mạng ngang hàng.

0.1.3 Mâu thuẫn cốt lõi của MLS trên mạng ngang hàng

Mâu thuẫn cốt lõi có thể được phát biểu như sau: MLS yêu cầu trạng thái nhóm tiến hóa theo chuỗi Epoch tuyến tính, trong khi mạng ngang hàng bất đồng bộ không tự cung cấp một thứ tự tuyến tính toàn cục cho các thông điệp thay đổi trạng thái. Khi hai thành viên cùng tạo Commit dựa trên Epoch E , mỗi Commit đều có thể là một ứng viên hợp lệ để đưa nhóm sang Epoch $E + 1$ nếu xét riêng tại thời

điểm nó được tạo. Tuy nhiên, hai Commit này thường tạo ra hai trạng thái cây khác nhau. Nếu một phần nhóm áp dụng Commit thứ nhất, trong khi phần còn lại áp dụng Commit thứ hai, toàn nhóm sẽ phân kỳ.

Trong mô hình tập trung, DS có thể xử lý xung đột bằng cách chỉ phát tán Commit thắng cuộc hoặc yêu cầu client chỉ chấp nhận Commit đầu tiên được DS xác nhận. Trong mô hình P2P, không có thực thể mặc định nào có quyền phá hòa. Do đó, nếu không có lớp điều phối bổ sung, MLS trên P2P có nguy cơ rơi vào trạng thái mà các nhánh khác nhau đều có vẻ hợp lệ cục bộ nhưng không thể giao tiếp liên nhánh.

Đây chính là khoảng trống mà đề án hướng tới: xây dựng một cơ chế điều phối phi tập trung để ảo hóa vai trò tuần tự hóa của DS, nhưng không đưa trở lại một máy chủ trung tâm và không sử dụng các thuật toán đồng thuận nặng nề cho mọi Commit.

0.2 Các kết quả nghiên cứu tương tự

Bài toán mã hóa đầu-cuối nhóm đã được tiếp cận theo nhiều hướng khác nhau. Các hướng này có thể chia thành bốn nhóm chính: mô hình Sender Keys dựa trên các kênh hai bên **perrin2025doubleratchet**, Megolm trong Matrix **matrix2026megolm**, MLS với Delivery Service tập trung **rfc9750**, và các nỗ lực phi tập trung hóa MLS **kohbrok2023dmls**, **alwen2023fork**.

0.2.1 Double Ratchet, Sender Keys và giới hạn khi mở rộng nhóm

Double Ratchet **perrin2025doubleratchet** là cơ chế nổi bật cho liên lạc hai bên, cung cấp khả năng thay đổi khóa liên tục theo từng thông điệp, nhờ đó đạt được bảo mật xuôi (Forward Secrecy - FS) và khả năng phục hồi sau thỏa hiệp (Post-Compromise Security - PCS) trong nhiều kịch bản. Khi mở rộng sang nhóm, một hướng phổ biến là sử dụng Sender Keys **whatsapp2026encryption**. Mỗi thành viên tạo một khóa gửi tin riêng và phân phối khóa này cho các thành viên còn lại thông qua các kênh bảo mật hai bên.

Mô hình Sender Keys có ưu điểm thực tiễn là sau khi khóa gửi tin đã được phân phối, người gửi có thể mã hóa một tin nhắn một lần rồi để hạ tầng phân phối bản mã đó tới các thành viên trong nhóm. Vì vậy, mô hình này hiệu quả hơn so với việc mã hóa riêng từng tin nhắn cho từng người nhận. Tuy nhiên, giới hạn lớn xuất hiện ở giai đoạn cập nhật khóa và phục hồi sau thỏa hiệp. Khi một sender key bị lộ, hệ thống cần tạo và phân phối khóa mới cho các thành viên còn lại. Chi phí này tăng theo kích thước nhóm đối với từng người gửi, và khi nhiều người gửi cần cập nhật khóa, tổng chi phí có thể trở thành gánh nặng đáng kể.

Điểm yếu quan trọng hơn là tính bảo mật sau thỏa hiệp (PCS). Nếu kẻ tấn công lấy được sender key của một thành viên, kẻ đó có thể tiếp tục nghe lén thụ động các thông điệp của thành viên đó cho đến khi khóa mới được sinh và phân phối lại. Đây là một trong những lý do MLS được đề xuất như một hướng tiếp cận có cấu trúc hơn cho mã hóa nhóm quy mô lớn.

0.2.2 Megolm và đánh đổi giữa hiệu năng và bảo mật sau thỏa hiệp

Megolm **matrix2026megolm** là cơ chế group ratchet được sử dụng trong hệ sinh thái Matrix. Thay vì thực hiện quá nhiều trao đổi khóa cho từng thông điệp, Megolm cho phép một phiên khóa được sử dụng để mã hóa chuỗi thông điệp trong nhóm. Cách tiếp cận này giúp giảm chi phí truyền thông và tính toán, đặc biệt trong các nhóm có nhiều tin nhắn.

Tuy nhiên, hiệu năng này đi kèm đánh đổi bảo mật. Khi trạng thái của một phiên Megolm bị lộ, kẻ tấn công có thể giải mã các thông điệp trong phạm vi phiên tương ứng, đặc biệt là các thông điệp từ điểm ratchet bị lộ trở về sau. Do đó, hệ thống cần định kỳ tạo phiên mới và phân phối lại khóa phiên để giới hạn phạm vi thiệt hại. Điều này cho thấy Megolm phù hợp với một số yêu cầu thực tiễn, nhưng chưa giải quyết triệt để bài toán PCS cho nhóm lớn và biến động thành viên thường xuyên.

0.2.3 MLS tập trung và vai trò của Delivery Service

MLS **rfc9420** được chuẩn hóa nhằm cung cấp cơ chế thiết lập khóa nhóm bất đồng bộ, hiệu quả và có bảo đảm bảo mật mạnh hơn cho nhóm nhiều thành viên. MLS sử dụng cấu trúc cây mật mã để giảm chi phí cập nhật khóa xuống theo logarit kích thước nhóm. Thay vì mỗi thay đổi phải gửi lại khóa đến toàn bộ thành viên theo cách tuyến tính, thành viên cập nhật chỉ cần thay đổi một đường đi trong cây và mã hóa bí mật mới cho các nhánh liên quan.

Trong các triển khai thông thường, MLS được đặt trong một hệ thống nhắn tin có Delivery Service **rfc9750**. DS có hai vai trò thực tế rất quan trọng. Thứ nhất, DS lưu trữ và cung cấp vật liệu khóa công khai ban đầu như KeyPackage, giúp các thành viên có thể thêm người khác vào nhóm ngay cả khi người đó đang offline. Thứ hai, DS phân phối MLS messages giữa các client. Với DS nhất quán mạnh, hệ thống còn có thể dựa vào DS để áp đặt thứ tự tuyến tính cho các Commit và phá hòa khi nhiều Commit được gửi cùng lúc.

Chính ở đây xuất hiện giới hạn đối với mục tiêu của đề án. Mô hình khách - chủ (Client-Server) truyền thống bộc lộ điểm yếu là phụ thuộc hoàn toàn vào hạ tầng trung tâm, tạo ra nguy cơ từ chối dịch vụ, rò rỉ metadata và khó triển khai trong môi trường mạng cô lập. Vì vậy, mặc dù RFC không bắt buộc DS phải là một máy chủ vật lý duy nhất, trong thực tế vận hành, phần lớn các kiến trúc MLS thông thường

vẫn cần một thành phần tương đương DS để giải quyết phân phối và thứ tự. Loại bỏ thành phần này khỏi kiến trúc không chỉ là thay đổi hạ tầng mạng, mà là loại bỏ bộ tuần tự hóa trung tâm của toàn bộ tiến trình cập nhật trạng thái nhóm.

0.2.4 Phi tập trung hóa MLS: đồng thuận phân tán và DMLS

Một hướng tiếp cận trực tiếp để đưa MLS lên P2P là sử dụng một cơ chế đồng thuận phân tán nhằm thống nhất thứ tự Commit. Các thuật toán đồng thuận như Raft, PBFT hoặc các hạ tầng blockchain có thể tạo ra một nhật ký lưu trữ (log) tuyến tính cho các thay đổi trạng thái nhóm. Khi mọi Commit được đưa vào cùng một log, các thành viên có thể áp dụng chúng theo cùng một thứ tự và tránh phân nhánh.

Tuy nhiên, cách tiếp cận này có chi phí cao. Đồng thuận phân tán thường yêu cầu nhiều vòng trao đổi mạng, có độ trễ lớn và phụ thuộc vào giả định đa số (quorum). Trong môi trường P2P có sự biến động nút mạng (peer churn), thiết bị di động, mạng yếu hoặc phân mảnh vật lý, việc duy trì quorum ổn định có thể không phù hợp. Đặc biệt, với mục tiêu ứng dụng nội bộ nhẹ, hoạt động được trong mạng cục bộ LAN hoặc mạng cách ly, việc đưa một lớp BFT hoặc blockchain vào chỉ để sắp xếp Commit có thể làm hệ thống trở nên nặng nề hơn bản thân bài toán cần giải quyết.

Một hướng khác là Decentralized MLS (DMLS), đề xuất mở rộng MLS để xử lý Commit ngoài thứ tự trong môi trường phi tập trung. Đề xuất DMLS của Konrad Kohbrok **kohbrok2023dmls** và các hướng như FREEK (Fork-Resilient Continuous Group Key Agreement) **alwen2023fork** cố gắng cho phép thành viên xử lý nhiều nhánh cập nhật và hội tụ lại sau đó. Ưu điểm của hướng này là tiếp cận bài toán ngay trong lớp mật mã và hướng tới khả năng chống fork tốt hơn. Tuy nhiên, việc xử lý Commit ngoài thứ tự thường đòi hỏi lưu giữ thêm vật liệu khóa cũ hoặc quản lý nhiều trạng thái tạm thời, từ đó tạo ra đánh đổi với Forward Secrecy và tăng độ phức tạp triển khai.

Từ các giới hạn trên, đề án lựa chọn hướng tiếp cận trung gian: không sửa đổi lõi MLS và không sử dụng đồng thuận nặng nề, mà xây dựng một lớp điều phối ứng dụng gồm bốn cơ chế: Single-Writer, Epoch Checks, Fork Healing và HLC Message Ordering. Lớp này cố gắng giảm xác suất concurrent commits trong điều kiện mạng liên thông, phát hiện phân nhánh khi network partition xảy ra, và hỗ trợ phục hồi trạng thái khi mạng kết nối trở lại.

0.2.5 So sánh tổng hợp các hướng tiếp cận

Bảng 1 tổng hợp các hướng tiếp cận chính đối với bài toán mã hóa nhóm và vận hành MLS trong môi trường phân tán.

Tiêu chí	Sender Keys / Pairwise	Megolm	MLS với DS tập trung	Consensus / Blockchain	DMLS / FREEK	Hướng đề xuất
Kiến trúc vận hành	Client-Server	Client-Server / Matrix	Thường có DS	P2P có đồng thuận	P2P mở rộng MLS	P2P với lớp điều phối
Chi phí cập nhật nhóm	Tăng theo số thành viên	Thấp hơn nhờ session	Logarit theo TreeKEM	Phụ thuộc đồng thuận	Logarit nhưng quản lý nhiều trạng thái	Logarit ở lõi MLS, thêm chi phí điều phối nhẹ
Tuần tự hóa	Không phải trọng tâm chính	Không phải trọng tâm chính	DS phá hòa	Log đồng thuận	Xử lý Commit ngoài thứ tự	Single-Writer và Fork Healing
FS / PCS	PCS khó cho nhóm lớn	PCS bị đánh đổi	FS/PCS mạnh theo MLS	Phụ thuộc MLS bên dưới	Có đánh đổi do giữ khóa cũ	Hướng tới giữ FS/PCS của MLS, có phân tích giới hạn
Hoạt động khi partition	Phụ thuộc server	Phụ thuộc server	DS có thể trở thành điểm nghẽn	Chỉ phân vùng có quorum	Có khả năng xử lý fork	Cho phép phân vùng tiến hóa có kiểm soát và phục hồi
Độ phức tạp triển khai	Trung bình	Trung bình	Cao nhưng chuẩn hóa	Rất cao	Rất cao	Cao ở lớp điều phối, không sửa lỗi MLS

Bảng 1: Bảng so sánh tổng hợp các hướng tiếp cận bài toán mã hóa và vận hành MLS phân tán

Bảng 1 cho thấy khoảng trống mà đề án khai thác: cần một cơ chế đủ nhẹ cho P2P, đủ chặt để bảo vệ trạng thái MLS, nhưng không biến hệ thống nhàn tin thành một hệ thống blockchain hoặc BFT đầy đủ.

0.3 Nền tảng MLS và TreeKEM

0.3.1 Các khái niệm cốt lõi: Client, Group, Member và Epoch

Trong MLS **rfc9420**, client là thực thể nắm giữ khóa mật mã và tham gia vào quá trình thiết lập trạng thái nhóm. Group là một tập logic gồm các client cùng chia sẻ một trạng thái mật mã tại một thời điểm. Member là client đang nằm trong trạng thái chia sẻ của group và có quyền truy cập các bí mật (secrets) của group. Epoch là một phiên bản cụ thể của trạng thái nhóm, trong đó tập thành viên hợp lệ cùng nắm giữ trạng thái mật mã tương ứng.

Điểm quan trọng là trạng thái group trong MLS không phải một tập dữ liệu rời rạc có thể hợp nhất tùy ý. Nó là một chuỗi Epoch tuyến tính, trong đó mỗi Epoch phụ thuộc vào Epoch trước đó. Mỗi thay đổi thành viên hoặc cập nhật khóa làm phát sinh một Epoch mới. Vì vậy, tính đúng đắn của MLS phụ thuộc mạnh vào việc tất cả thành viên cùng nhìn nhận một chuỗi Epoch tương thích.

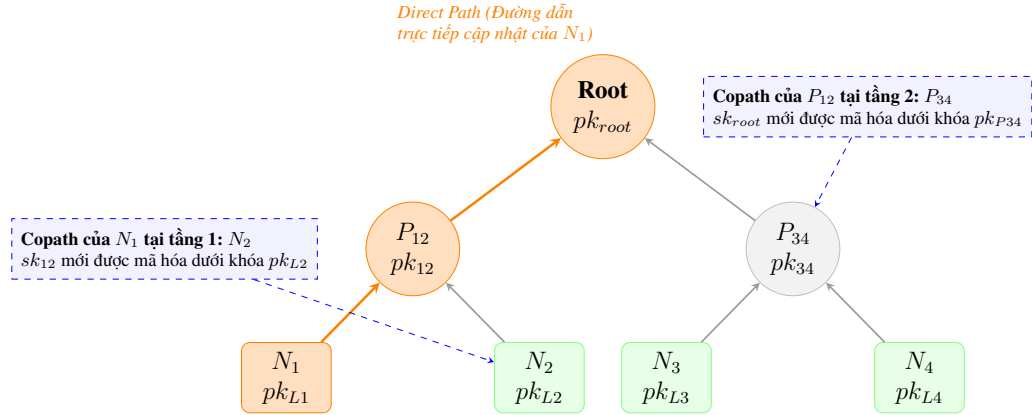
0.3.2 Ratchet Tree và TreeKEM

TreeKEM là cơ chế trung tâm giúp MLS đạt hiệu quả khi cập nhật khóa nhóm. Thay vì mọi thành viên phải trao đổi khóa với mọi thành viên còn lại, MLS tổ chức các thành viên thành lá của một cây nhị phân gọi là Ratchet Tree. Các nút trong cây chứa vật liệu khóa công khai, còn các thành viên biết một số bí mật tương ứng với đường đi từ lá của mình lên gốc.

Khi một thành viên cập nhật khóa, thành viên đó tạo một đường dẫn cập nhật

(update path) mới từ lá của mình lên gốc cây. Các bí mật mới trên đường đi này được mã hóa cho các nhánh đồng cấp tương ứng (copath), sao cho các thành viên còn lại có thể giải mã và cập nhật trạng thái. Nhờ cấu trúc cây, số lượng thao tác mã hóa và kích thước thông tin cập nhật tăng theo chiều cao cây, tức xấp xỉ logarit theo số thành viên trong nhóm ($O(\log N)$).

Sơ đồ cấu trúc cây Ratchet Tree và quá trình cập nhật đường dẫn (Path Update) được mô tả chi tiết trong Hình 0.2.



Hình 0.2: Cấu trúc cây nhị phân Ratchet Tree và tiến trình cập nhật đường dẫn (Path Update) của thành viên N_1

Cần lưu ý rằng bí mật tại gốc cây (sk_{root}) không được sử dụng trực tiếp như khóa mã hóa tin nhắn ứng dụng. Sau mỗi Commit, MLS đưa các bí mật mới qua lịch trình sinh khóa (key schedule) để tạo ra các bí mật của Epoch mới, bao gồm các khóa dùng cho xác thực, mã hóa thông điệp, xuất dữ liệu khóa (exporter) và các cơ chế liên quan. Vì vậy, TreeKEM nên được hiểu là cơ chế cập nhật trạng thái khóa nhóm, còn việc sinh khóa cụ thể cho thông điệp ứng dụng (application messages) được thực hiện bởi key schedule của MLS.

0.3.3 Proposal, Commit và tiến trình chuyển Epoch

MLS phân biệt rõ Proposal và Commit. Proposal là thông điệp đề xuất một thay đổi đối với nhóm, ví dụ thêm thành viên (Add), loại bỏ thành viên (Remove), cập nhật khóa (Update) hoặc thay đổi phần mở rộng của bối cảnh nhóm (group context). Proposal tự nó chưa làm thay đổi Epoch. Commit là thông điệp thực sự áp dụng một tập Proposal và đưa nhóm từ Epoch hiện tại sang Epoch mới.

Khi tạo Commit, thành viên thực hiện Commit lựa chọn một tập Proposal hợp lệ, áp dụng chúng lên Ratchet Tree và group context, tạo đường dẫn cập nhật (update path) nếu cần, sau đó sinh ra trạng thái mật mã mới. Các thành viên nhận Commit sẽ kiểm tra chữ ký, kiểm tra điều kiện hợp lệ của danh sách Proposal, áp dụng thay đổi theo đúng quy tắc, tính toán trạng thái mới và xác minh thể xác nhận

(confirmation tag). Confirmation tag đóng vai trò xác nhận rằng các thành viên đã đi tới cùng một trạng thái nhóm sau khi xử lý Commit.

Từ góc nhìn hệ phân tán, Commit là thao tác ghi vào trạng thái mật mã nhóm. Vì mỗi Commit tạo ra một Epoch mới, việc có nhiều Commit cạnh tranh tại cùng một Epoch chính là nguồn gốc của phân nhánh trạng thái. Do đó, lớp điều phối của đồ án tập trung kiểm soát quyền tạo Commit, trong khi vẫn cho phép nhiều thành viên gửi Proposal.

0.3.4 Tree hash, transcript hash và epoch authenticator

Để phát hiện sự đồng bộ hoặc phân kỳ giữa các thành viên, MLS sử dụng nhiều giá trị ràng buộc trạng thái. Tree hash phản ánh cấu trúc Ratchet Tree hiện tại. Transcript hash ràng buộc lịch sử các thông điệp bắt tay (handshake) đã được xác nhận. Epoch authenticator là giá trị có thể được dùng để xác nhận rằng các thành viên đang chia sẻ cùng trạng thái Epoch.

Các giá trị này rất quan trọng đối với đồ án. Trong môi trường P2P, hai nút có thể cùng báo rằng chúng thuộc cùng một định danh nhóm (group_id) và cùng một số Epoch, nhưng thực chất tree hash hoặc transcript hash khác nhau. Khi đó, hai nút không thật sự ở cùng trạng thái mật mã. Vì vậy, cơ chế Fork Healing ở Chương 3 cần dựa trên các định danh trạng thái như epoch, tree hash, commit hash hoặc epoch authenticator để phát hiện phân nhánh.

0.3.5 Delivery Service và bài toán tuần tự hóa Commit

Delivery Service trong kiến trúc MLS **rfc9750** có thể được hiểu là lớp dịch vụ cung cấp hai chức năng chính: lưu trữ và cung cấp vật liệu khóa ban đầu, đồng thời phân phối các thông điệp MLS giữa các client. Trong các triển khai Client-Server thông thường, DS còn thường được dùng như một điểm tuần tự hóa tự nhiên cho các thông điệp thay đổi trạng thái nhóm.

Về mặt tiêu chuẩn, MLS không bắt buộc DS phải là một máy chủ trung tâm duy nhất. DS có thể được hiện thực bởi một hạ tầng tập trung, một hệ thống liên miền hoặc thậm chí bởi chính mạng ngang hàng nếu các client truyền các thông điệp MLS qua P2P. Tuy nhiên, khi không có DS tập trung, các client phải tự cung cấp những thuộc tính mà DS thường đảm nhiệm, đặc biệt là phân phối thông điệp, xử lý Commit đến trễ, phát hiện Commit cạnh tranh và hòa giải trạng thái.

Có thể nói, trong triển khai MLS thông thường, DS là “người gác cổng thời gian” của trạng thái nhóm: nó giúp tất cả thành viên nhìn thấy các thay đổi theo một thứ tự đủ nhất quán. Khi loại bỏ DS khỏi kiến trúc, hệ thống không chỉ mất một máy chủ truyền tin, mà còn mất điểm sắp xếp thứ tự trung tâm. Đây là lý do

lớp điều phối tập trung trở thành thành phần bắt buộc trong thiết kế của đồ án.

0.3.6 Application messages, delayed messages và giới hạn Forward Secrecy

MLS phân biệt handshake messages và application messages. Handshake messages như Proposal và Commit làm thay đổi hoặc chuẩn bị thay đổi trạng thái nhóm. Application messages là thông điệp ứng dụng được mã hóa bằng khóa sinh ra từ trạng thái Epoch hiện tại.

Trong mạng thực tế, application messages có thể đến muộn hoặc đến sai thứ tự. MLS có cơ chế xử lý ở mức bộ khóa của người gửi (sender ratchet) và bộ đếm thế hệ (generation counter) để hỗ trợ một mức độ nhất định đối với thông điệp bị trễ hoặc bị sắp xếp lại. Tuy nhiên, để bảo vệ Forward Secrecy, client không nên giữ khóa cũ vô thời hạn. Điều này tạo ra một đánh đổi quan trọng: nếu xóa khóa cũ quá sớm, các thông điệp đến muộn có thể không giải mã được; nếu giữ khóa cũ quá lâu, phạm vi thiệt hại khi thiết bị bị thỏa hiệp sẽ tăng.

Đánh đổi này là nền tảng cho cơ chế Epoch Checks và bộ đệm thông điệp trong đồ án. Khi một nút nhận thông điệp thuộc Epoch cũ, Epoch hiện tại hoặc Epoch tương lai, lớp điều phối phải quyết định loại bỏ, xử lý, đệm tạm thời hay kích hoạt đồng bộ trạng thái. Chính sách này cần cân bằng giữa tính sẵn sàng của hệ thống, khả năng đồng bộ ngoại tuyến và yêu cầu bảo mật xuôi.

0.3.7 External Commit và cơ chế gia nhập lại nhóm

MLS hỗ trợ cơ chế gia nhập từ bên ngoài (external join) thông qua External Commit, cho phép một client bên ngoài nhóm tạo Commit để tự thêm mình vào nhóm nếu có thông tin nhóm (GroupInfo) hợp lệ và thỏa mãn các chính sách của ứng dụng. Đây là cơ chế quan trọng đối với đồ án vì trong quá trình Fork Healing, các nút ở nhánh thua có thể cần hủy trạng thái cũ và gia nhập lại nhánh thắng.

Trong ngữ cảnh của đề tài, External Join không được dùng như một thao tác tùy tiện. Nó là bước phục hồi có kiểm soát sau khi hệ thống xác định rằng trạng thái cục bộ của một nút không còn thuộc nhánh được chọn. Nút đó cần dừng sử dụng khóa cũ cho thông điệp mới, lấy trạng thái công khai cần thiết của nhánh thắng, thực hiện quy trình gia nhập lại và chỉ tiếp tục gửi thông điệp sau khi có trạng thái MLS hợp lệ mới.

0.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán

0.4.1 Đặc tính của mạng ngang hàng

Mạng ngang hàng khác với mô hình Client-Server ở chỗ không có một máy chủ trung tâm duy nhất điều phối toàn bộ luồng truyền thông. Các nút có thể tự phát

hiện nhau, thiết lập kết nối trực tiếp hoặc gián tiếp, và trao đổi dữ liệu qua nhiều đường truyền. Đặc tính này giúp hệ thống giảm phụ thuộc vào hạ tầng trung tâm, nhưng đồng thời làm tăng độ phức tạp của điều phối.

Trong mạng P2P, các vấn đề phổ biến gồm địa chỉ IP thay đổi, NAT, tường lửa, thiết bị ngoại tuyến (offline), sự biến động nút mạng (peer churn), độ trễ không đồng đều, trùng lặp thông điệp (message duplication) và phân mảnh mạng (network partition). Với ứng dụng thông thường, các vấn đề này có thể chỉ gây trễ hoặc mất dữ liệu tạm thời. Với MLS, chúng có thể làm phát sinh lỗi nghiêm trọng hơn: một số nút chuyển Epoch trước, một số nút vẫn ở Epoch cũ, hoặc hai phân vùng mạng cùng tạo Commit khác nhau.

Do đó, lớp mạng P2P chỉ giải quyết bài toán truyền thông. Nó không tự giải quyết bài toán nhất quán trạng thái mật mã. Bất kỳ hệ thống MLS-over-P2P nào cũng cần một lớp điều phối phía trên để quyết định khi nào được tạo Commit, Commit nào được chấp nhận, thông điệp nào cần đệm và trạng thái nào là trạng thái hợp lệ của nhóm.

0.4.2 go-libp2p và các thành phần truyền thông

Thư viện go-libp2p **libp2p2026doc** là một thư viện mạng ngang hàng mô-đun hóa, cung cấp các thành phần quan trọng để xây dựng ứng dụng P2P. Mỗi nút trong libp2p có một mã định danh peer (PeerID) được dẫn xuất từ khóa công khai, nhờ đó danh tính mạng không phụ thuộc hoàn toàn vào địa chỉ IP. Các nút có thể được định vị thông qua multiaddr, cho phép mô tả địa chỉ kết nối qua nhiều giao thức và tầng mạng khác nhau.

Để thiết lập kết nối, libp2p hỗ trợ nhiều giao thức truyền vận (transport) như TCP, QUIC hoặc WebRTC tùy môi trường triển khai. Sau khi kết nối, các peer có thể nâng cấp kênh truyền lên kênh bảo mật bằng các giao thức như Noise hoặc TLS. Libp2p cũng hỗ trợ cơ chế đa luồng (stream multiplexing) để nhiều luồng ứng dụng chạy trên cùng một kết nối, giúp hệ thống có thể vừa gửi thông điệp chat, vừa đồng bộ trạng thái hoặc truyền tệp.

Về phát hiện peer, libp2p cung cấp nhiều cơ chế như mDNS cho mạng cục bộ LAN, bootstrap peer, rendezvous hoặc Kademlia DHT cho môi trường rộng hơn. Với ứng dụng của đề án, mDNS phù hợp cho kịch bản mạng nội bộ, còn DHT hoặc bootstrap có thể hỗ trợ các kịch bản WAN/VPN.

0.4.3 GossipSub và giới hạn về thứ tự thông điệp

GossipSub **libp2p2026doc** là cơ chế publish/subscribe phổ biến trong libp2p, cho phép các nút tham gia vào một chủ đề (topic) và phát tán thông điệp qua mạng

lưới liên kết (mesh). Khi một peer gửi thông điệp vào topic, thông điệp được lan truyền qua các peer lân cận và tiếp tục được truyền phát (gossip) tới các peer khác. Cơ chế này phù hợp để phát tán Proposal, Commit, nhịp đập mạng (heartbeat) hoặc thông báo trạng thái nhóm trong môi trường P2P.

Tuy nhiên, GossipSub không cung cấp các bảo đảm mà MLS cần cho việc tuân thủ hóa Commit. Nó không bảo đảm mọi peer nhận thông điệp theo cùng một thứ tự, không bảo đảm phân phối đúng một lần (exactly-once delivery) và không tự quyết định thông điệp nào là Commit thắng cuộc khi có xung đột. GossipSub là lớp truyền bá thông điệp, không phải lớp đồng thuận trạng thái.

Giới hạn này là lý do đồ án không thể chỉ “đưa MLS lên GossipSub” là đủ. Nếu chỉ phát tán Commit qua GossipSub mà không có cơ chế điều phối, hệ thống vẫn có nguy cơ concurrent commits và cryptographic state fork. Vì vậy, các cơ chế Single-Writer, Epoch Checks và Fork Healing được đặt phía trên GossipSub như một lớp kiểm soát trạng thái.

0.4.4 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng

Trong MLS, Epoch chỉ thay đổi khi có thao tác thay đổi trạng thái nhóm, chẳng hạn Add, Remove, Update hoặc Commit. Trong cùng một Epoch, nhiều thành viên có thể gửi application messages song song. Các thông điệp này không nhất thiết cần một thứ tự mật mã toàn cục giống Commit, nhưng ứng dụng chat vẫn cần một thứ tự hiển thị ổn định và hợp lý giữa các thiết bị.

Đồng hồ vật lý không đủ tin cậy trong mạng P2P vì các thiết bị có thể lệch giờ, không có NTP hoặc bị cấu hình sai. Vì vậy, hệ phân tán thường sử dụng các loại đồng hồ logic để biểu diễn quan hệ thứ tự giữa sự kiện. Lamport Clock cung cấp thứ tự logic đơn giản; Vector Clock biểu diễn quan hệ nhân quả chi tiết hơn nhưng có chi phí tăng theo số nút; Hybrid Logical Clock (HLC) **kulkarni2014hlc** kết hợp thời gian vật lý và bộ đếm logic để tạo timestamp vừa gần với thời gian thực, vừa ổn định hơn khi có lệch giờ.

Trong đồ án, HLC được sử dụng cho thứ tự hiển thị application messages, không dùng để quyết định tính hợp lệ của Commit. Nói cách khác, HLC giúp người dùng trên các thiết bị khác nhau nhìn thấy dòng chat theo thứ tự nhất quán hơn, nhưng không thay thế cơ chế MLS Epoch, không thay thế chữ ký, không thay thế confirmation tag và không giải quyết concurrent commits. Việc tách bạch này rất quan trọng: Commit cần cơ chế điều phối trạng thái; application messages cần cơ chế sắp xếp hiển thị.

0.4.5 Khoảng trống cần giải quyết

Từ các nền tảng trên, có thể thấy bài toán của đề tài nằm ở giao điểm giữa mật mã học nhóm và hệ phân tán. MLS cung cấp lõi mật mã mạnh nhưng giả định hệ thống triển khai phải giải quyết phân phối và thứ tự thông điệp. go-libp2p cung cấp hạ tầng truyền thông ngang hàng nhưng không cung cấp thứ tự tuyến tính cho Commit. GossipSub giúp phát tán thông điệp nhưng không phải cơ chế đồng thuận. HLC giúp sắp xếp application messages nhưng không bảo vệ trạng thái mật mã.

Khoảng trống nghiên cứu của đề án chính là thiết kế một lớp điều phối phi tập trung nằm giữa hai thế giới này. Lớp điều phối phải đủ nhẹ để phù hợp với P2P, đủ chặt để bảo vệ chuỗi Epoch tuyến tính của MLS, và đủ thực dụng để phục hồi khi mạng bị chia cắt. Bốn cơ chế được đề xuất ở Chương 3 — Single-Writer, Epoch Checks, Fork Healing và HLC Message Ordering — được xây dựng trực tiếp từ khoảng trống này.

0.5 Kết chương

Chương 2 đã trình bày nền tảng lý thuyết cho bài toán vận hành MLS trên mạng ngang hàng. Trước hết, chương làm rõ rằng thách thức chính của đề tài không chỉ là truyền thông điệp qua P2P, mà là duy trì tính nhất quán trạng thái mật mã của một nhóm MLS khi không còn máy chủ trung tâm đóng vai trò tuần tự hóa. Tiếp theo, chương phân tích các hướng tiếp cận hiện có như Sender Keys, Megolm, MLS tập trung, đồng thuận phân tán và DMLS, qua đó chỉ ra khoảng trống giữa hiệu năng, bảo mật và khả năng vận hành phi tập trung.

Chương cũng trình bày các khái niệm cốt lõi của MLS và TreeKEM, bao gồm Group, Epoch, Proposal, Commit, Delivery Service, tree hash, transcript hash, application messages và External Commit. Những khái niệm này là cơ sở để hiểu vì sao concurrent commits có thể gây rẽ nhánh trạng thái mật mã và vì sao việc phục hồi sau phân mảnh mạng cần được xử lý cẩn trọng. Cuối cùng, chương phân tích nền tảng go-libp2p, GossipSub và HLC, đồng thời nhấn mạnh rằng lớp mạng ngang hàng chỉ cung cấp khả năng truyền thông, không tự bảo đảm thứ tự tuyến tính hay đồng thuận trạng thái.

Các phân tích trên tạo tiền đề trực tiếp cho Chương 3, nơi đề án trình bày chi tiết giao thức điều phối phi tập trung gồm bốn cơ chế: Single-Writer để giảm concurrent commits, Epoch Checks để bảo vệ tính đơn điệu của trạng thái cục bộ, Fork Healing để phục hồi khi mạng bị phân mảnh, và HLC Message Ordering để sắp xếp thông điệp ứng dụng trong cùng một Epoch.